

# Multimodal Visual Understanding

## Multimodal Visual Understanding

1. Course Content
2. Starting the Agent
3. Running the Case
  - 3.1 Starting the Program
  - 3.2 Test Cases
    - 3.2.1 Case 1
4. Source Code Analysis

## 1. Course Content

Basics: Run the example program to allow the robot to observe its environment and execute task instructions.

Advanced: Understanding the key source code introduced in this lesson

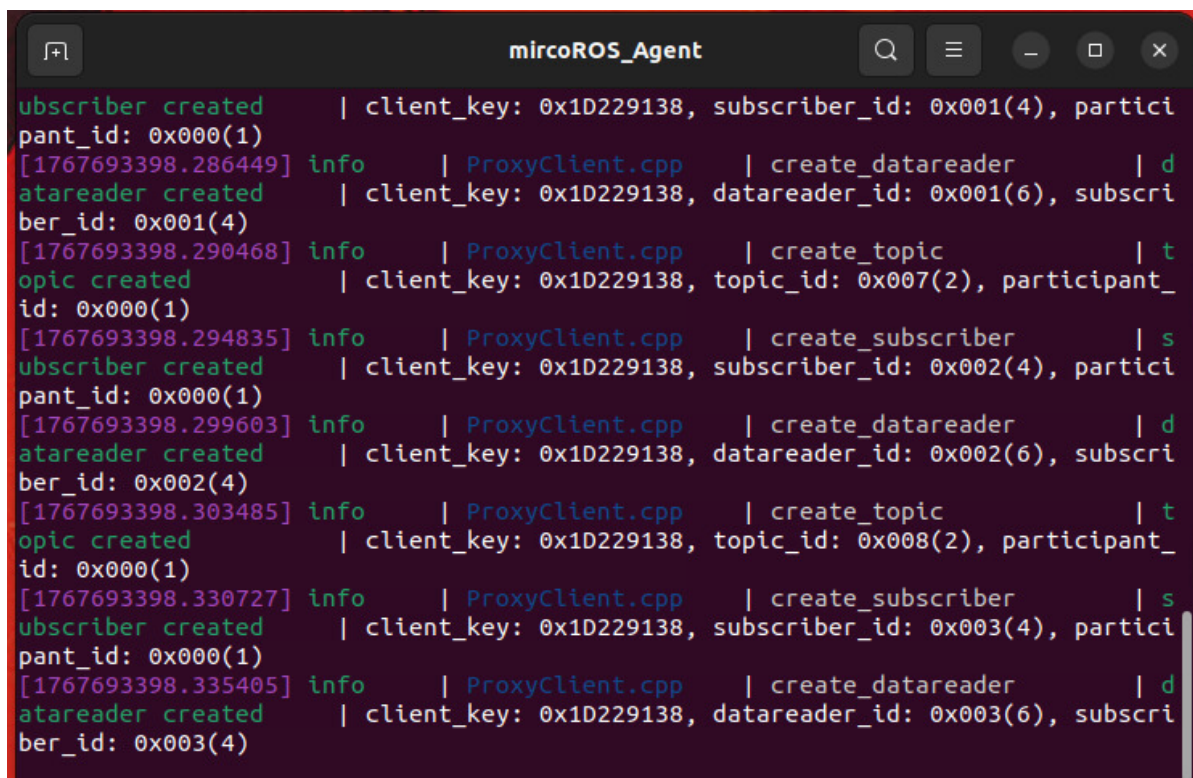
## 2. Starting the Agent

**Note: The agent must be started before testing any case. If it is already started, it does not need to be started again.**

Enter the following command on the vehicle's terminal:

```
start_agent
```

The terminal prints the following information, indicating a successful connection:



```
subscriber created      | client_key: 0x1D229138, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1767693398.286449] info    | ProxyClient.cpp      | create_datareader      | d
atareader created      | client_key: 0x1D229138, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1767693398.290468] info    | ProxyClient.cpp      | create_topic            | t
opic created           | client_key: 0x1D229138, topic_id: 0x007(2), participant_
id: 0x000(1)
[1767693398.294835] info    | ProxyClient.cpp      | create_subscriber      | s
ubscriber created      | client_key: 0x1D229138, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1767693398.299603] info    | ProxyClient.cpp      | create_datareader      | d
atareader created      | client_key: 0x1D229138, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
[1767693398.303485] info    | ProxyClient.cpp      | create_topic            | t
opic created           | client_key: 0x1D229138, topic_id: 0x008(2), participant_
id: 0x000(1)
[1767693398.330727] info    | ProxyClient.cpp      | create_subscriber      | s
ubscriber created      | client_key: 0x1D229138, subscriber_id: 0x003(4), partici
pant_id: 0x000(1)
[1767693398.335405] info    | ProxyClient.cpp      | create_datareader      | d
atareader created      | client_key: 0x1D229138, datareader_id: 0x003(6), subscri
ber_id: 0x003(4)
```

## 3. Running the Case

---

### 3.1 Starting the Program

For Raspberry Pi 5 users, if the ROS container is not running, you need to start it first.

```
bringup_m3
```

Open a terminal inside the container.

```
exec_m3
```

Open the terminal on the vehicle's terminal and enter the following command:

```
ros2 launch m3_bringup llm_agent_control.launch.py
```

Alternatively, you can use the following shortcut:

```
multi_brains
```

### 3.2 Test Cases

The test cases here are for demonstration purposes only. Users can create their own dialogue commands.

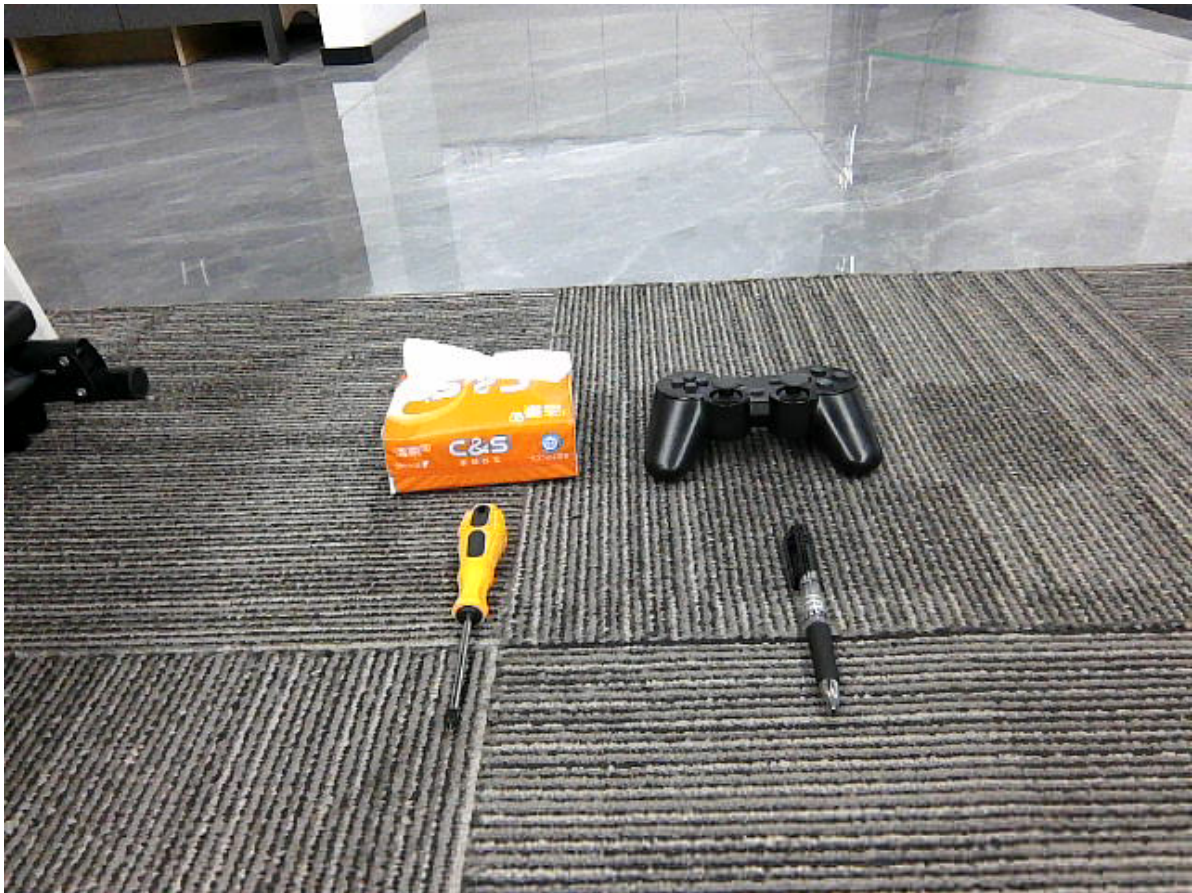
- Help me see what items are in front of me and explain their functions.

#### 3.2.1 Case 1

- First, wake up the robot with "Hello yahboom". The robot will respond. After hearing the recording prompt tone, the user can speak. If there is sound activity, it will print "1-1-1-1"; if there is no sound activity, it will print "-----". After speaking, it will perform an end-of-speech detection. If there is silence for more than 1.5 seconds, recording will stop.
- The robot will first respond to the user, then follow the instructions. Simultaneously, the terminal will print the following information:

```
jetson@yahboom: ~  
[asr-5] [INFO] [1749289605.035907489] [asr]: 0  
[asr-5] [INFO] [1749289605.125912424] [asr]: 0  
[asr-5] [INFO] [1749289605.217016024] [asr]: 0  
[asr-5] [INFO] [1749289605.308455466] [asr]: 0  
[asr-5] [INFO] [1749289605.398672531] [asr]: 0  
[asr-5] [INFO] [1749289605.459957154] [asr]: 0  
[asr-5] [INFO] [1749289605.549669799] [asr]: 0  
[asr-5] [INFO] [1749289605.640377363] [asr]: 0  
[asr-5] [INFO] [1749289607.353032474] [asr]: 请你帮我看一看你面前都有什么物品，并和我讲讲他们都有什么作用。  
[asr-5] [INFO] [1749289607.354477637] [asr]: okay😊, let me think for a moment...  
[model_service-3] [INFO] [1749289612.628742482] [model_service]: The upcoming task to be carried out: 1. 使用视觉传感器扫描面前的物品，识别并记录下所有物品的名称。  
[model_service-3] 2. 对于每一件识别出的物品，查询其相关信息或用途。  
[model_service-3] 3. 将识别出的每件物品及其作用依次告知用户。  
[model_service-3] [INFO] [1749289615.794415750] [model_service]: "action": ['seewhat()'], "response": 好的，我现在就用我的眼睛看看面前都有些什么物品  
[model_service-3] [INFO] [1749289634.474911787] [model_service]: "action": [], "response": 我看到了面前有以下物品：一盒纸巾、一把螺丝刀、一个游戏手柄和一支笔。让我来告诉你它们各自的作用吧！  
[model_service-3]  
[model_service-3] 1. **纸巾**：这盒纸巾可以用来擦拭汗水、灰尘或者清洁物品，非常实用。  
[model_service-3] 2. **螺丝刀**：螺丝刀是用来拧紧或松开螺丝的工具，常用于组装或修理各种设备。  
[model_service-3] 3. **游戏手柄**：这是一个游戏控制器，通常用于玩电子游戏，提供更舒适和精准的操作体验。  
[model_service-3] 4. **笔**：这支笔可以用来书写或绘画，是学习和工作的必备工具。  
[model_service-3]  
[model_service-3] 这些物品各有各的用途，是不是很有趣？  
[model_service-3] [INFO] [1749289703.891385820] [model_service]: "action": ['finishtask()'], "response": 我已经介绍完所有物品及其作用啦，如果你还有其他需要帮忙的，请随时告诉我哦！
```

Robot's perspective:



[!IMPORTANT]

Important Note:

- The robot possesses short-term memory. After being activated, all information is stored in the AI's historical context memory. The robot will only clear its previous memory when the user actively requests to end the current task or instructs the robot to step down or rest.

## 4. Source Code Analysis



- Robot Action Source Code Path:

```
~/yahboomM3_ws/code_ws/src/multi_brains/multi_brains/action_service.py
```

- Model Service Source Code:

```
~/yahboomM3_ws/code_ws/src/multi_brains/multi_brains/model_service.py
```

- The main program implementing the robot's visual observation function is the `seewhat` method in the `action_service.py` program:
- The function implementation is: save and display an image from the latest perspective
- Then send a request to the `model_service` node of the model service, requesting the multi\_brains agent in the dify to provide feedback on the image.

```
def seewhat(self):
    """
    Save the current view image and send it back to the dify-agent.
    """
    self.save_single_image()
    msg=LlmRequest()
    msg.llm_request=self.actionlog.get_text("image_feedback")
    msg.robot_feedback=True
    self.llm_request_pub.publish(msg)
    return None

def save_single_image(self):
    """保存图片 / Save a single image"""
    cv_image = self.bridge.imgmsg_to_cv2(self.image_msg, "bgr8")
    cv2.imwrite(self.image_cache_path, cv_image)
    time.sleep(0.05)
    display_thread = threading.Thread(target=self.__display_saved_image)
    display_thread.start()

def __display_saved_image(self):
    """
    显示已保存的图片4秒后关闭窗口 / Display the saved image for 4 seconds before
    closing the window
    """
    try:
        img = cv2.imread(self.image_cache_path)
        if img is not None:
            cv2.imshow("Saved Image", img)
            cv2.waitKey(4000) # 等待4秒 / wait for 4 seconds
            cv2.destroyAllWindows()
        else:
            self.get_logger().error(
                "Failed to load saved image for display."
            ) # Loading saved image for display failed...
    except Exception as e:
        self.get_logger().error(f"Error displaying image: {e}") # Error
        displaying image...
```

- Additionally, in `model_service.py`, the `llm_request_callback` callback function is used to receive requests to access the `multi_brains` agent.
- If the `llm_request` field in the request information is an image request, then a list `[msg.llm_request, 'image_request', True]` is constructed and added to the model request processing queue.

```
def llm_request_callback(self, msg:LlmRequest):
    """
    话题回调函数,接收调用模型请求并放入队列中 / Topic callback function, receive
    model request and put into queue
    """
    if self.debug_mode: self.get_logger().info(f"robot_feedback:
{msg.robot_feedback},llm_request:{msg.llm_request}")
    if msg.robot_feedback:
        # Robot Feedback Request
        if msg.llm_request ==self.syslog.get_text("image_feedback"):

self.llm_handler_queue.put([msg.llm_request,'image_request',True])
        elif msg.llm_request == "finish":
            # Upon receiving the finish command from dify-agent, the current
            task cycle ends, which clears the historical context and begins a new task
            cycle.

            self.clear_request_queue() # Clear the request queue
            self.dify_llmclient.reset_conversation() # Reset Session
        else:
            #Standard robot feedback results
            if self.debug_mode:
self.get_logger().info(self.syslog.get_text("system_log_4"))

self.llm_handler_queue.put([msg.llm_request,'text_request',True])

        else:# Model requests from other sources
            self.llm_handler_queue.put([msg.llm_request,'text_request',None])
```